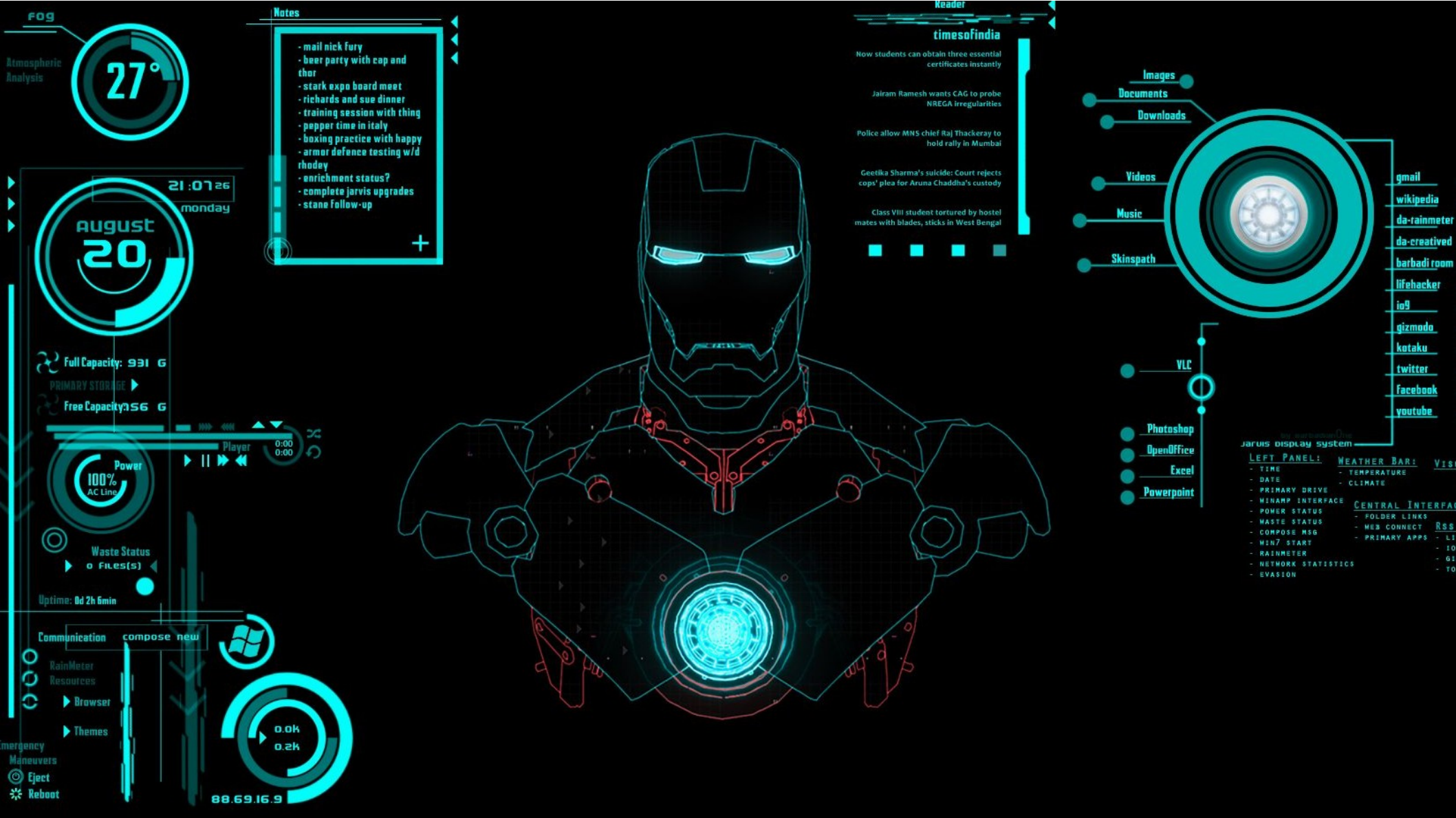




13.Câmera Virtual

Prof. Alexandre Krohn

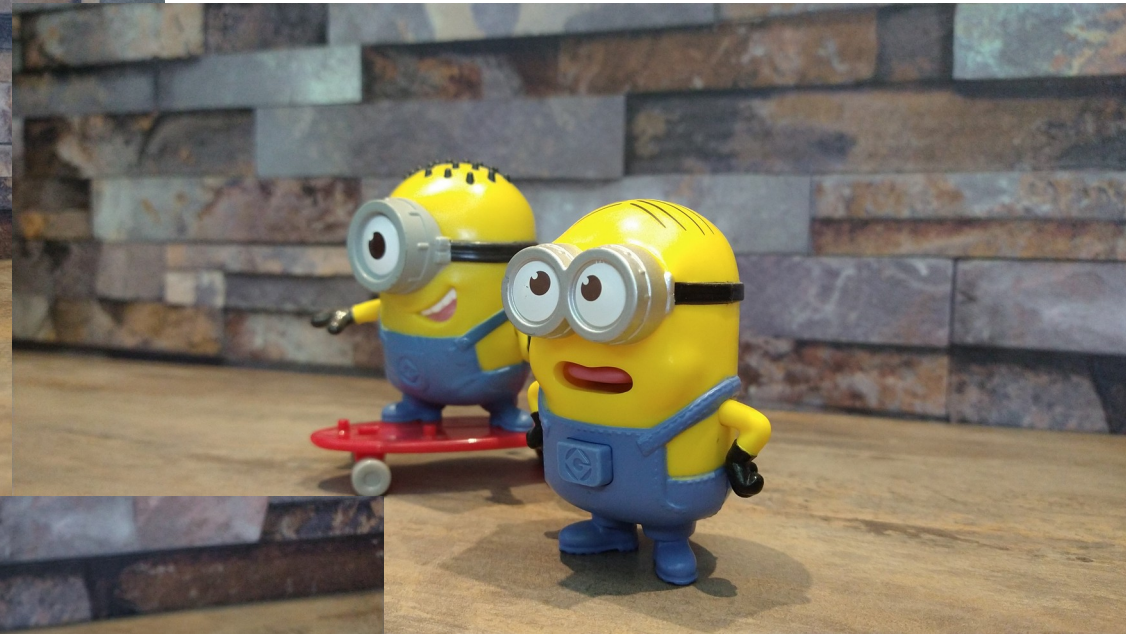
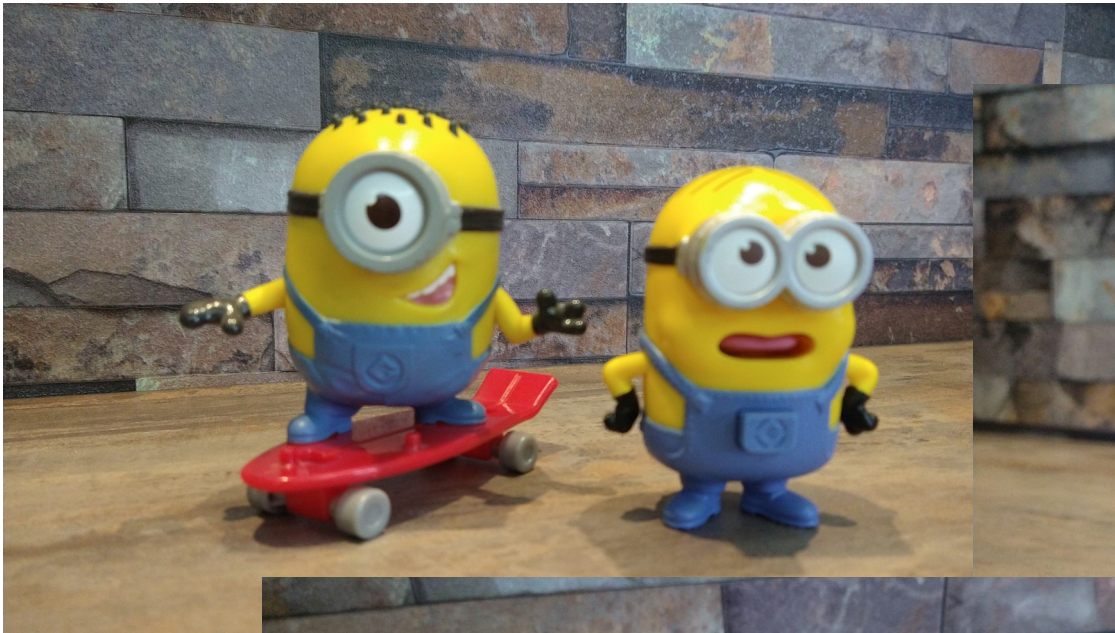
Roteiro

- Posicionamento de Objetos
- Câmera Virtual
- Matriz LookAt
- Movimentação da câmera

Posicionamento de Objetos

- Os mesmos objetos, posicionados no mesmo lugar, geram imagens diferentes ao ser observados de pontos de vista diferentes.

Posicionamento de Objetos

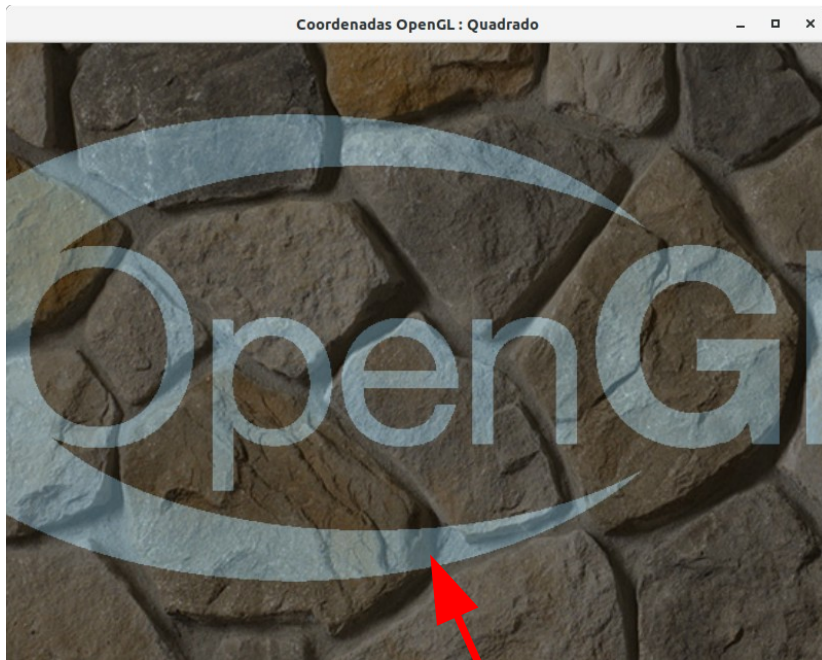
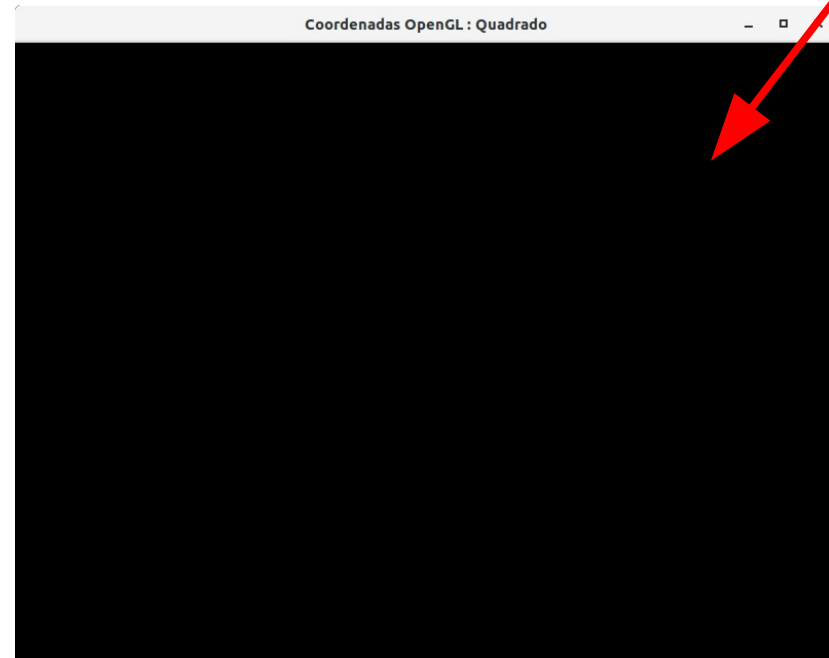


Posicionamento de Objetos

- Na aula passada, no exemplo **GLFW_Coordenadas_3D_Quadrado**, foi necessário afastar-se da imagem gerada para que a mesma tornasse-se visível
- Isso foi feito ajustando-se a matriz de visualização

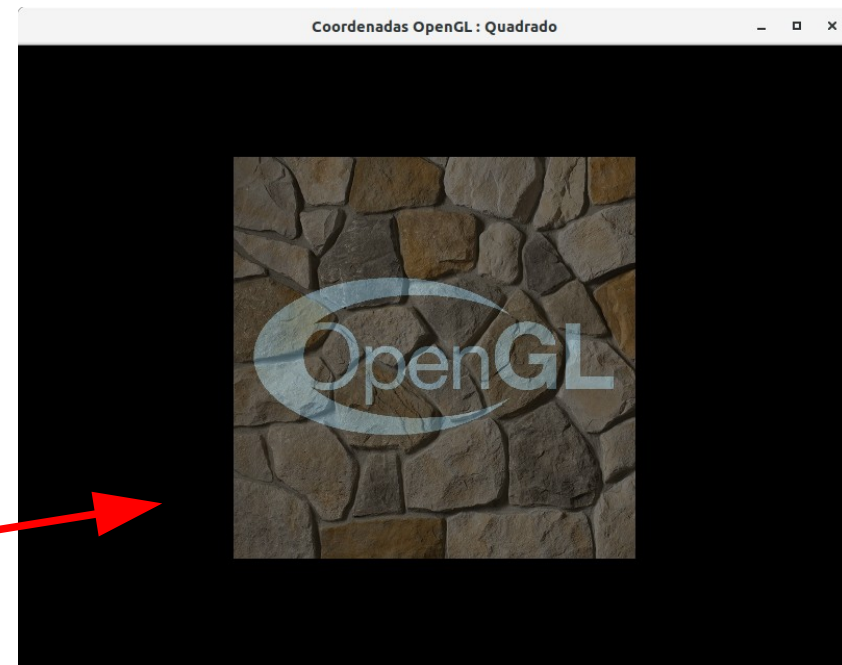
Posicionamento de Objetos

view original



```
view = glm::translate(view, glm::vec3(0.0f, 0.0f, -1.0f));
```

```
view = glm::translate(view, glm::vec3(0.0f, 0.0f, -3.0f));
```

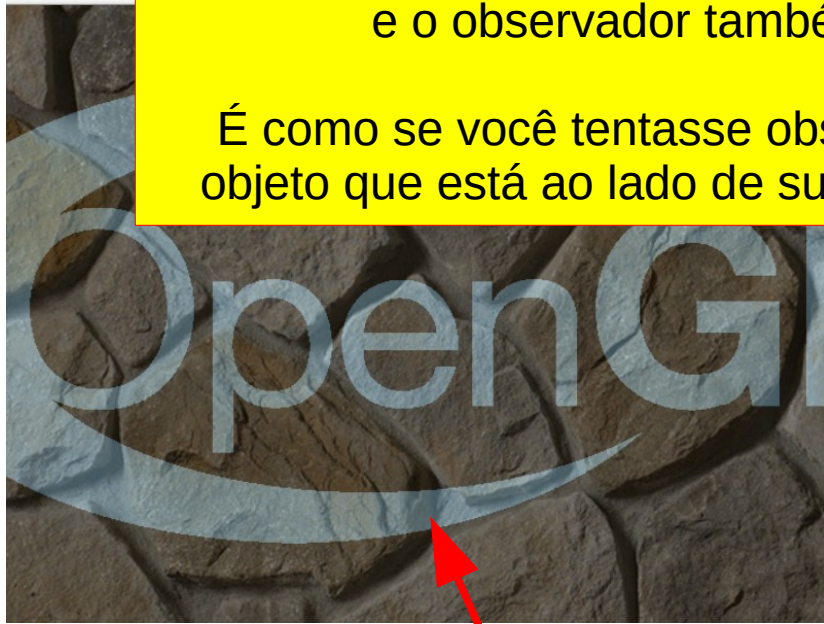


Posicionamento de Objetos

view original

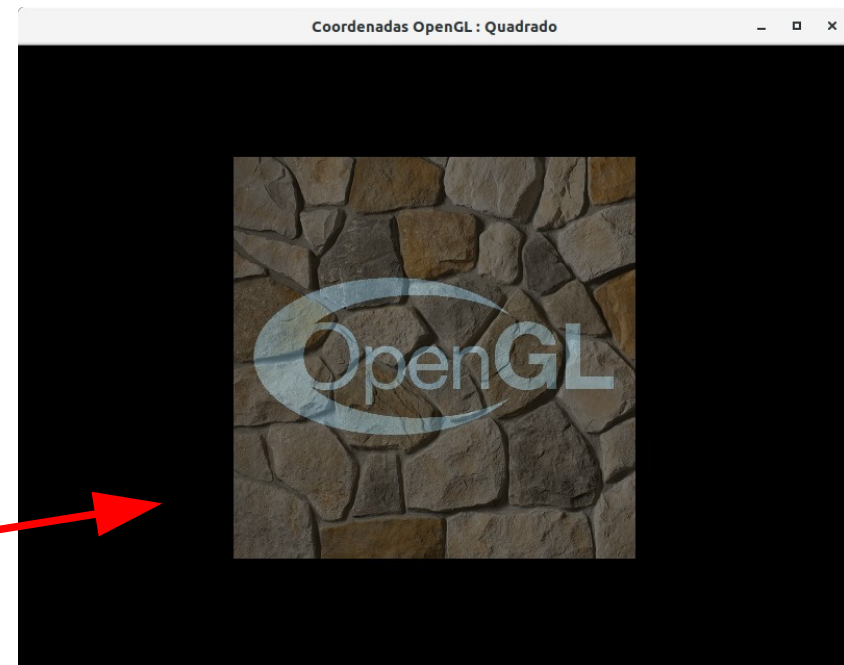
A imagem não é vista aqui porque
Está posicionada na coordenada $z = 0$,
e o observador também.

É como se você tentasse observar um
objeto que está ao lado de suas orelhas!



```
view = glm::translate(view, glm::vec3(0.0f, 0.0f, -1.0f));
```

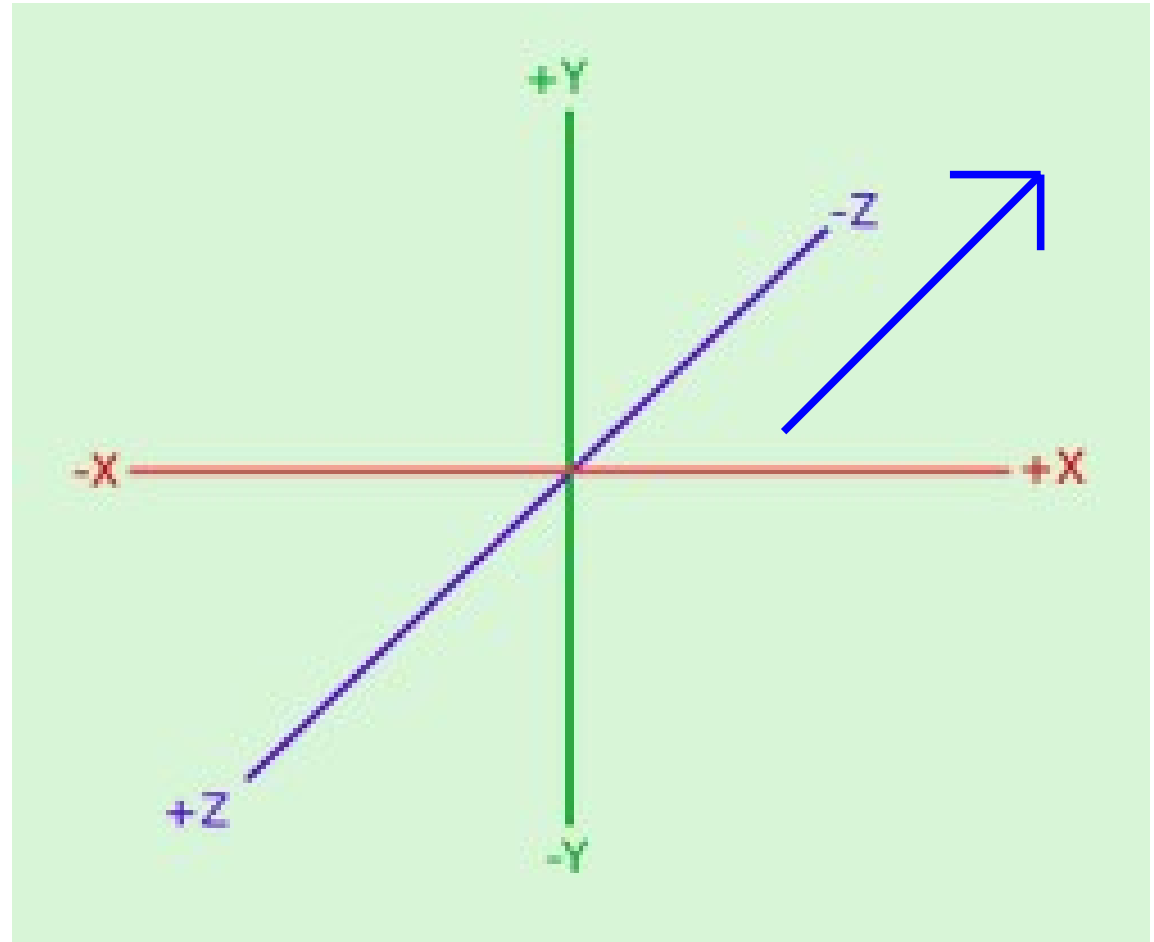
```
view = glm::translate(view, glm::vec3(0.0f, 0.0f, -3.0f));
```



Posicionamento de Objetos

Translação no sentido negativo do eixo Z, em 3 unidades:

Afasta o objeto observado 3 unidades



X Y Z

```
view = glm::translate(view, glm::vec3(0.0f, 0.0f, -3.0f));
```

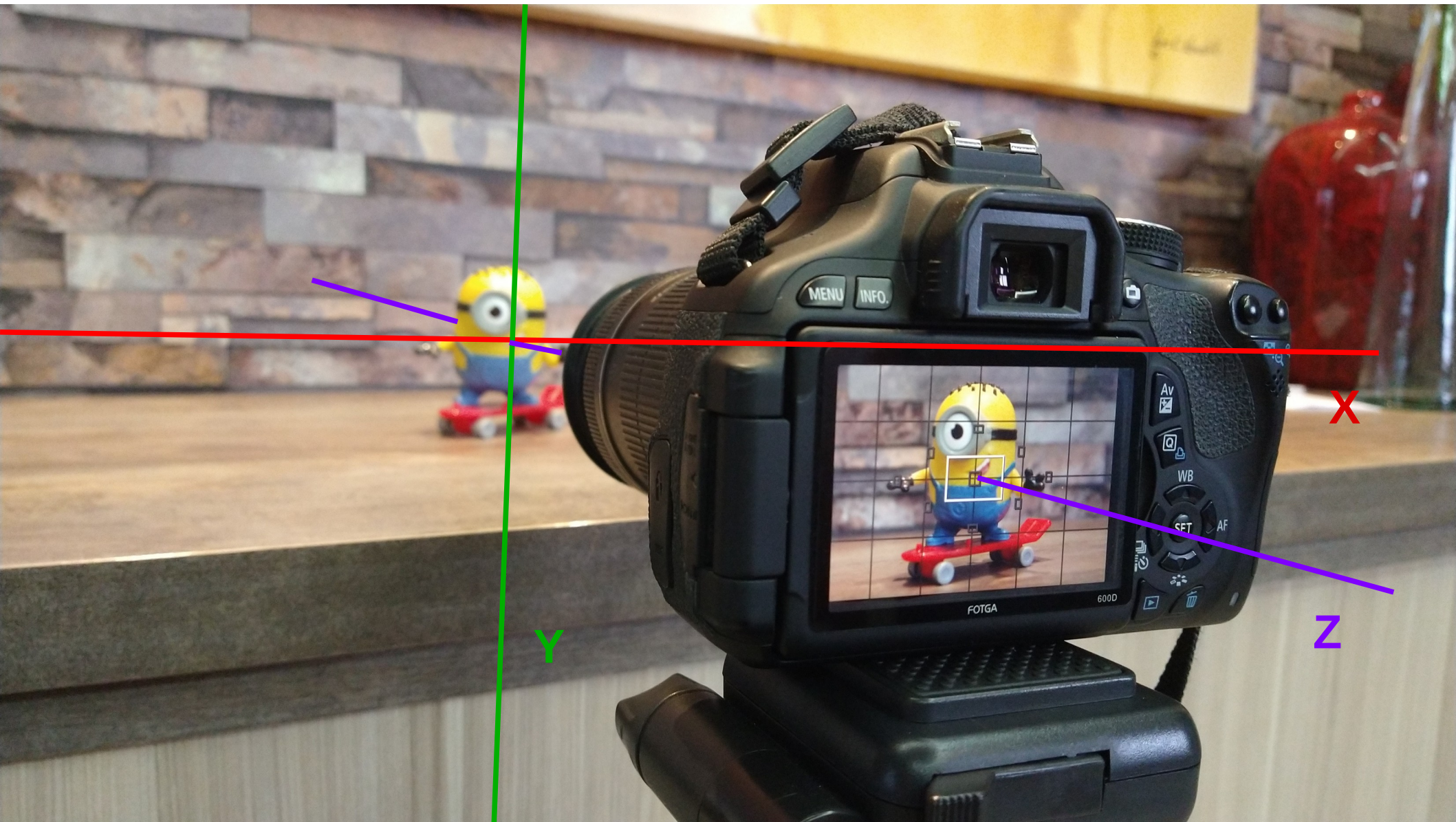

Roteiro

- Posicionamento de Objetos
- Câmera Virtual
- Matriz LookAt
- Movimentação da câmera

Câmera virtual

- A maneira como percebemos os objetos 3D é semelhante à maneira como uma câmera fotográfica converte imagens reais em fotografias.
- A câmera foi criada para simular nossa visão.
- Mover a câmera muda o ponto de vista, e portanto gera imagens diferentes.

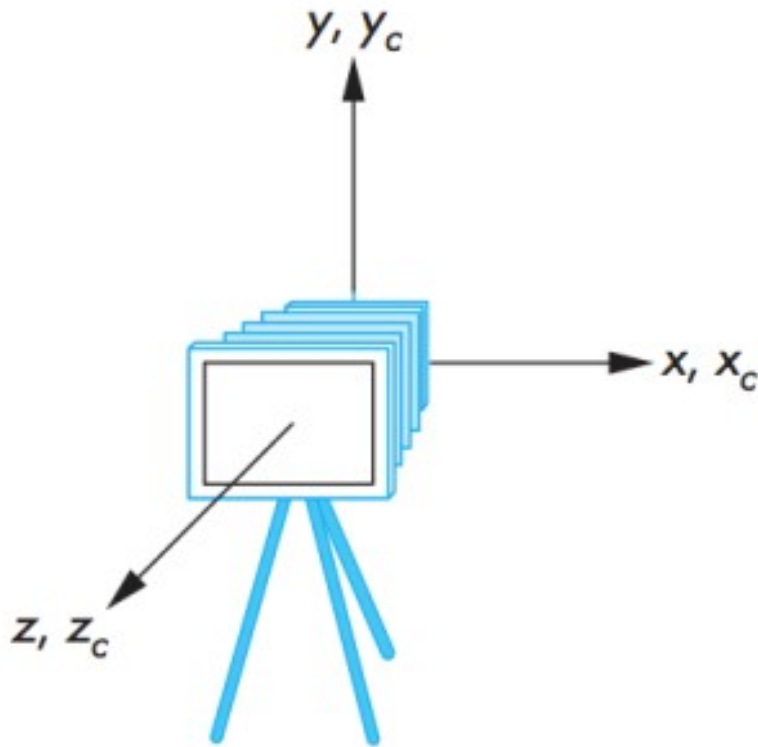
Câmera Virtual



Câmera virtual

- Em computação gráfica, utilizamos um modelo matemático para simular as alterações de ponto de vista.
- Isso é feito com matrizes de transformação

Câmera virtual



- Quando não configuramos nada em respeito à como vemos a image projetada na tela, estamos posicionados sobre o eixo **z**, olhando diretamente para a origem **(0,0,0)**. **Esse é o padrão da OpenGL**
- Por isso nossos objetos são mostrados centralizados na tela

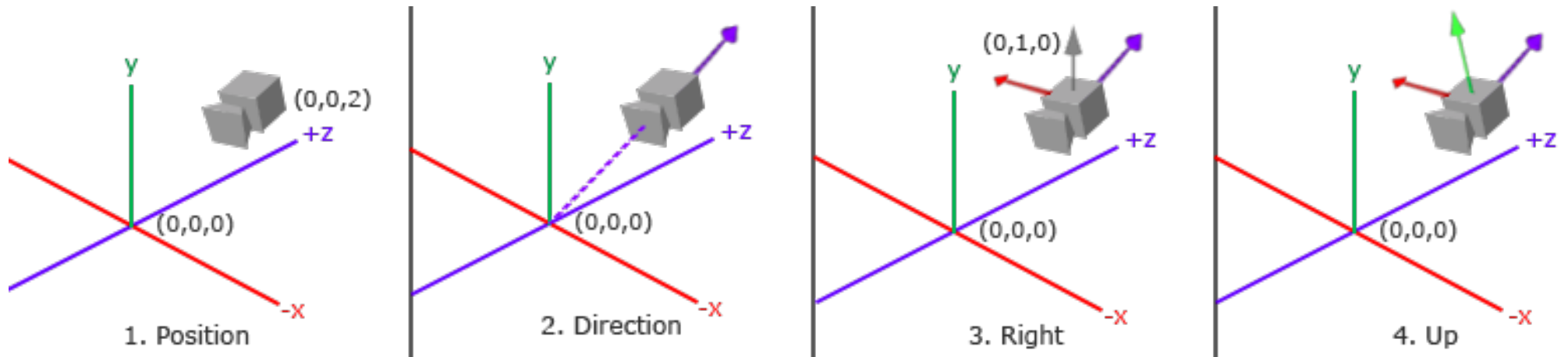
Câmera / Espaço de Visão

- Quando falamos sobre câmera / espaço de visão, estamos falando sobre todas as coordenadas do vértice vistas da perspectiva da câmera como a origem da cena: a **matriz de visão** transforma todas as coordenadas mundiais em coordenadas de visão que são **relativas à posição da câmera** e direção.

Câmera / Espaço de Visão

- Para definir uma câmera, precisamos de:
 - Sua posição no espaço mundial,
 - A direção para a qual está olhando,
 - Um vetor apontando para a direita e
 - Um vetor apontando para cima a partir da câmera (Eixo ascendente).
- Criaremos um sistema de coordenadas com 3 eixos de unidades perpendiculares com a posição da câmera como origem.

Câmera / Espaço de Visão



- 1) **Position** : Posição da câmera no espaço mundial
- 2) **Direction** : A direção para a qual está apontando
- 3) **Right** : Um vetor apontando para a direita
- 4) **Up** : Um vetor apontando para cima a partir da câmera

1) Posição da Câmera

- Obter a posição da câmera é feito da seguinte forma:
 - A posição da câmera é um vetor no espaço mundial que aponta para a posição da câmera. Colocamos a câmera na mesma posição que a colocamos no exemplo da aula anterior:

```
glm::vec3 cameraPos = glm::vec3(0.0f, 0.0f, -3.0f);
```

Não podemos esquecer de que o eixo z positivo está passando pela tela em sua direção, então, **se quisermos que a câmera se mova para trás, movemos ao longo do eixo z positivo.**

2) Direção da Câmera

- O próximo vetor necessário é a direção da câmera, por exemplo, em que direção está apontando. Por enquanto, deixamos a câmera apontar para a origem da nossa cena: $(0,0,0)$.
 - Subtrair o vetor de posição da câmera do vetor de origem da cena resulta no vetor de direção que queremos.

```
glm::vec3 cameraTarget = glm::vec3(0.0f, 0.0f, 0.0f);  
glm::vec3 cameraDirection = glm::normalize(cameraPos - cameraTarget);
```

Na verdade o vetor de direção aponta na direção **inversa** de seu alvo, vindo dele para a câmera

3) Eixo direito (Right)

- O próximo vetor de que precisamos é um vetor direito que represente o eixo x positivo do espaço da câmera.
 - Para obter o vetor correto, usamos um pequeno truque, primeiro especificando um vetor para cima que aponta para cima(óbvio) (no espaço mundial).
 - Em seguida, fazemos um produto vetorial no vetor superior e o vetor de direção da etapa 2.
 - Como o resultado de um produto vetorial é um vetor perpendicular a ambos os vetores, obteremos um vetor que aponta na direção do eixo x positivo

```
glm::vec3 up = glm::vec3(0.0f, 1.0f, 0.0f);  
glm::vec3 cameraRight = glm::normalize(glm::cross(up, cameraDirection));
```

A função **glm::cross(..)** realiza o produto vetorial entre dois vetores.
O resultado precisa ser normalizado para ficar entre 0 e 1.

4) Eixo Ascendente (Up)

- Agora que temos o vetor do eixo x e o vetor do eixo z, recuperar o vetor que aponta para o eixo y positivo da câmera é relativamente fácil:
 - Calculamos o produto vetorial do vetor direito e de direção:

```
glm::vec3 cameraUp = glm::cross(cameraDirection, cameraRight);
```

Em álgebra linear, esses quatro
Passos são chamados de
Processo de Gram-Schmidt

Roteiro

- Posicionamento de Objetos
- Câmera Virtual
- **Matriz LookAt**
- Movimentação da câmera

Matriz LookAt

- Definindo um espaço de coordenadas usando 3 eixos perpendiculares (ou não lineares), pode-se criar uma matriz com esses 3 eixos mais um vetor de translação, e a partir daí pode-se transformar qualquer vetor para esse espaço de coordenadas **multiplicando-o por esta matriz.**

Matriz LookAt

$$\text{LookAt} = \begin{bmatrix} R_x & R_y & R_z & 0 \\ U_x & U_y & U_z & 0 \\ D_x & D_y & D_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & -P_x \\ 0 & 1 & 0 & -P_y \\ 0 & 0 & 1 & -P_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **R** é o vetor direito,
- **U** é o vetor para cima,
- **D** é o vetor de direção e
- **P** é o vetor de posição da câmera

Matriz LookAt

- Observe que as partes de rotação (matriz esquerda) e translação (matriz direita) são invertidas - transpostas e negadas respectivamente - pois queremos girar e transladar o mundo na direção oposta de para onde queremos que a câmera se mova.

Matriz LookAt

- Usar a matriz **LookAt** como nossa matriz de visualização efetivamente transforma todas as coordenadas mundiais para o espaço de visualização que acabamos de definir.
- A matriz **LookAt** faz o que seu nome diz : ela é uma matriz de visualização que olha para um determinado alvo.

glm::lookAt(..)

- GLM possui uma função que cria a matriz **LookAt**, simplificando em muito o processo:

```
glm::mat4 view;  
view = glm::lookAt(glm::vec3(0.0f, 0.0f, 3.0f),  
                  glm::vec3(0.0f, 0.0f, 0.0f),  
                  glm::vec3(0.0f, 1.0f, 0.0f));
```

- A função recebe **três parâmetros**, sendo cada um deles um vetor:
 - Posição da câmera
 - Direção do alvo (posição para onde a câmera aponta)
 - Vetor para cima (indica qual lado da câmera está para cima)

Roteiro

- Posicionamento de Objetos
- Câmera Virtual
- Matriz LookAt
- Movimentação da câmera

Movimentação de câmera

- Movimentação de câmera pode ser feita de diversas formas.
 - Girar a câmera ao redor do objeto
 - “Caminhar” pela cena
 - “Olhar” em volta
 - Erguer-se ou Abaixar-se

Girar a câmera

- A rotação **automática** da câmera ao redor do objeto pode ser obtida da seguinte forma:

```
glm::mat4 view;
```

```
const float radius = 2.0f;
```

```
float camX = sin(glm::getTime()) * radius;
```

```
float camZ = cos(glm::getTime()) * radius;
```

```
view = glm::lookAt(glm::vec3(camX, 1.0, camZ), glm::vec3(0.0, 0.0, 0.0),  
    glm::vec3(0.0, 1.0, 0.0));
```

- A função **glfwGetTime()** fornece o relógio da aplicação. Utilizá-la significa calcular uma nova matriz de visualização a cada frame desenhado

Caminhar pela cena

- Girar em volta do objeto pode ser divertido, mas não nos oferece controle algum sobre o cenário
- Para podermos aproveitar mais nosso cenário, precisamos de liberdade para nos movimentarmos por ele, controlando nossa posição através de inputs do teclado

Caminhar pela cena

- Para poder caminhar pela cena, primeiro, **precisamos configurar um sistema de câmera**, e para isso é necessário definir algumas variáveis de câmera no topo do nosso programa (**globais**):

```
glm::vec3 cameraPos   = glm::vec3(0.0f, 0.0f, 3.0f);  
glm::vec3 cameraFront = glm::vec3(0.0f, 0.0f, -1.0f);  
glm::vec3 cameraUp    = glm::vec3(0.0f, 1.0f, 0.0f);
```

- A chamada da função **lookAt** fica:

```
view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);
```

Caminhar pela cena

- Primeiro, definimos a posição da câmera para os `cameraPos` previamente definidos.
- A direção é a posição atual + o vetor de direção que acabamos de definir. Isso garante que, independentemente de como nos movamos, a câmera continuará olhando na direção do alvo.
- Vamos agora atualizar o vetor **`cameraPos`** usando eventos de teclado

Caminhar pela cena

- Já possuímos uma função `processInput(..)` em nosso código, tratando os eventos de teclado (**ESC**). Vamos adicionar mais algumas linhas ao mesmo:

```
void processInput(GLFWwindow *window)
{
    // ...
    const float cameraSpeed = 0.05f;
    // ajustar de acordo com a velocidade do computador

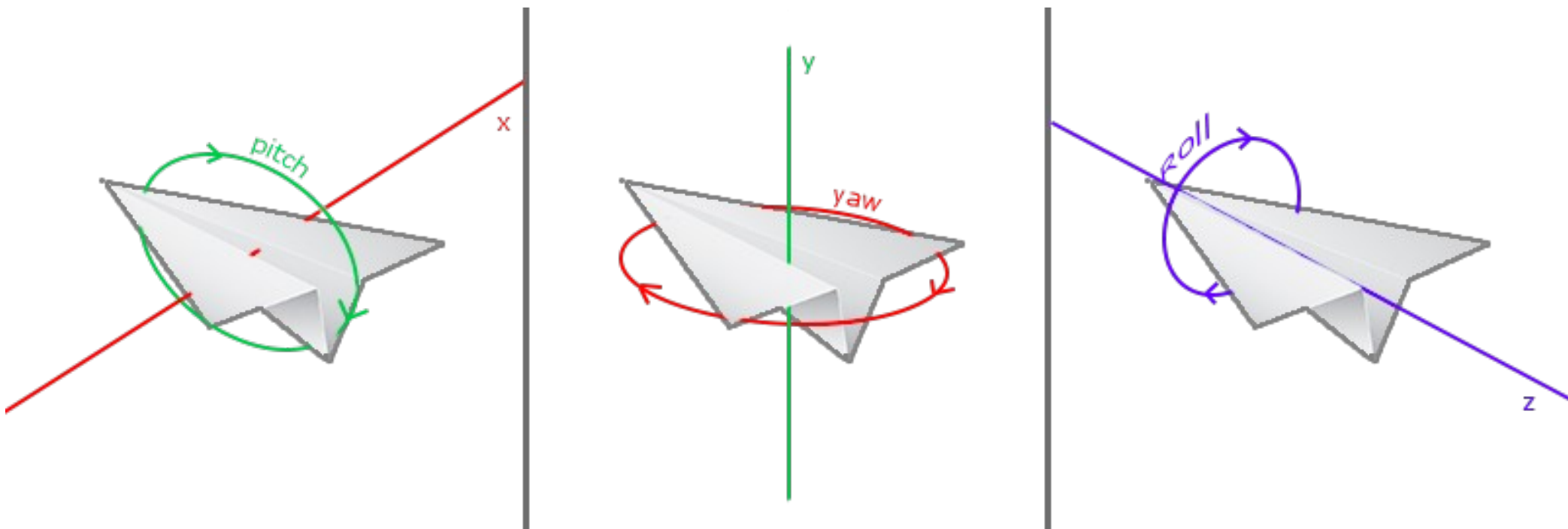
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        cameraPos += cameraSpeed * cameraFront;
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
        cameraPos -= cameraSpeed * cameraFront;
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
        cameraPos -= glm::normalize(glm::cross(cameraFront, cameraUp)) * cameraSpeed;
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
        cameraPos += glm::normalize(glm::cross(cameraFront, cameraUp)) * cameraSpeed;
}
```

Olhar em volta

- Para olhar ao redor da cena, temos que mudar o vetor **cameraFront** com base em outras teclas.
- No entanto, alterar o vetor de direção é um pouco complicado e requer alguma trigonometria.

Ângulos de Euler

- Os ângulos de **Euler** são 3 valores que podem representar qualquer rotação em 3D, definidos por **Leonhard Euler** por volta de 1700. Existem 3 ângulos de Euler: **pitch**, **yaw** e **roll**.



Ângulos de Euler

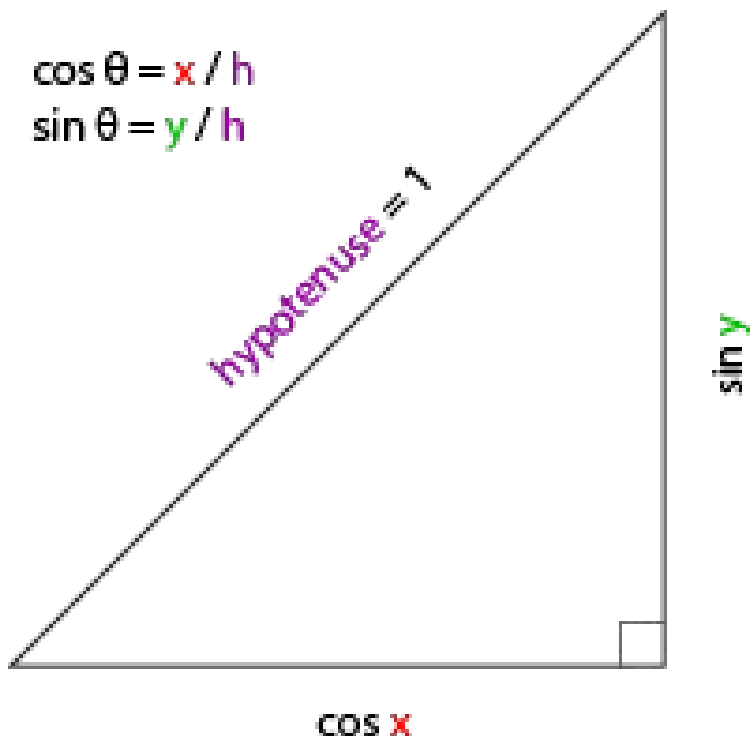
- A **inclinação**(pitch) é o ângulo que representa o quanto estamos olhando para cima ou para baixo, conforme visto na primeira imagem.
- O valor de **virada** (yaw) representa o quanto que estamos olhando para a esquerda ou para a direita.
- O valor de **rotação** (roll) representa o quanto rolamos, como usado principalmente em câmeras de jogos que simulam voo.
 - Cada um dos ângulos de Euler é representado por um único valor e com a combinação de todos os 3 deles podemos calcular qualquer vetor de rotação em 3D.

Olhar em volta

- Para nosso sistema de câmera, nos preocupamos apenas com os valores de **yaw** e **pitch**, então não discutiremos o valor de rotação aqui.
- Dados um valor de **pitch** e **yaw**, podemos convertê-los em um vetor 3D que representa um novo vetor de direção.

Fazendo algumas contas

- Imagine um triângulo retângulo:



- Se definirmos a hipotenusa como tendo comprimento 1, sabemos por trigonometria que o comprimento do lado adjacente é:

$$\cos x / h = \cos x / 1 = \cos x$$

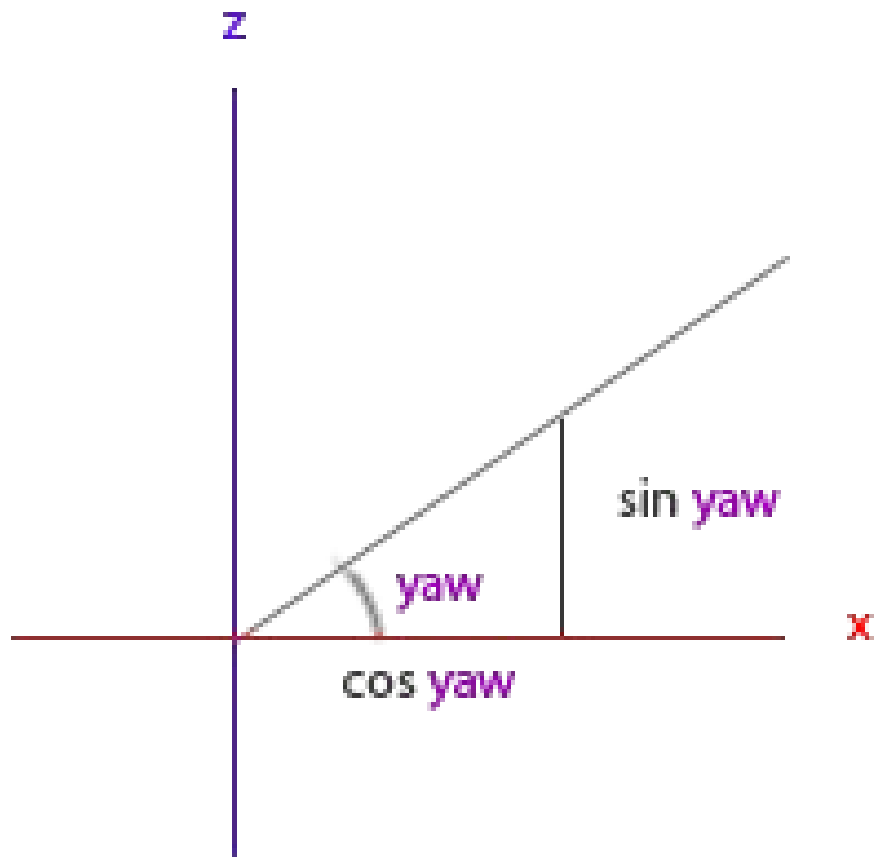
- e que o comprimento do lado oposto é

$$\sin y / h = \sin y / 1 = \sin y$$

- Isso nos dá algumas fórmulas gerais para recuperar o comprimento em ambos os lados x e y em triângulos retângulos, dependendo do ângulo fornecido.
- Vamos usar isso para calcular os componentes do vetor de direção.

Fazendo algumas contas

- Vamos imaginar esse mesmo triângulo, mas agora olhando para ele de uma perspectiva superior com os lados adjacentes e opostos sendo paralelos aos eixos x e z da cena (como se estivesse olhando para baixo no eixo y).



- Se visualizarmos o ângulo de virada como sendo o ângulo anti-horário começando do lado x , podemos ver que o comprimento do lado x está relacionado a $\cos(yaw)$.
- E da mesma forma como o comprimento do lado z se relaciona com o $\sin(yaw)$.

Determinando a virada da câmera

- Se tomarmos esse conhecimento e um determinado valor de virada, podemos usá-lo para criar um **vetor de direção da câmera**:

```
glm::vec3 direction;
```

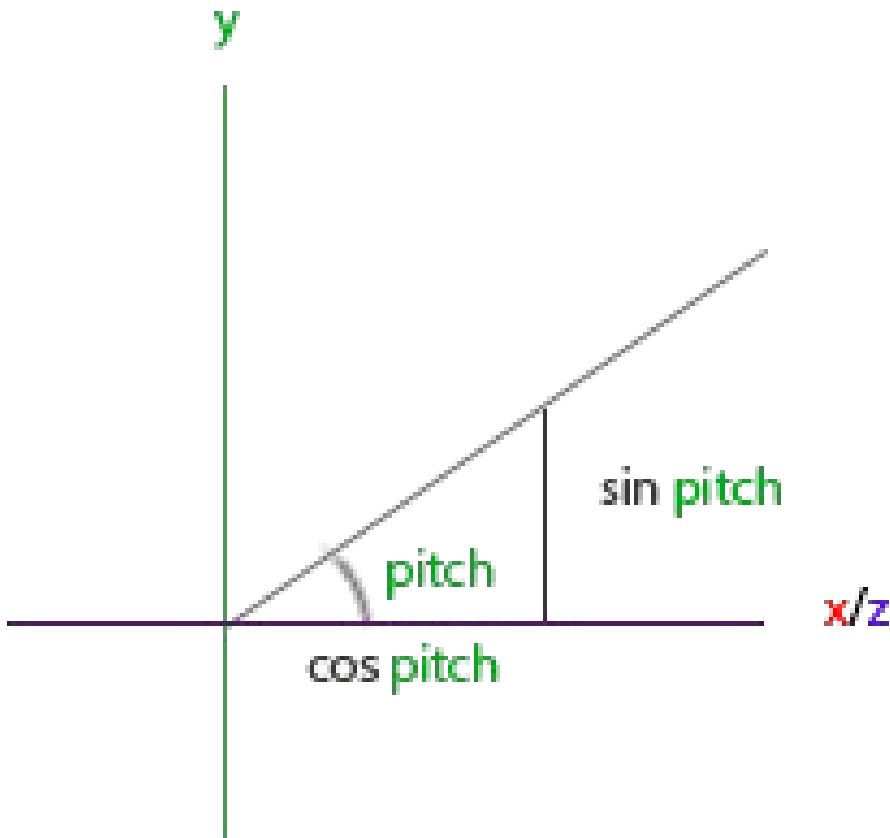
```
direction.x = cos(glm::radians(yaw));
```

```
// Note que é necessário converter o ângulo para radianos primeiro
```

```
direction.z = sin(glm::radians(yaw));
```

E a inclinação da câmera?

- Vamos agora olhar para o lado do eixo y como se estivéssemos sentados no plano xz:



- Da mesma forma, a partir deste triângulo, podemos ver que o **componente y** da direção é igual a **sin (pitch)**, portanto:

`direction.y = sin(glm::radians(pitch));`

E a inclinação da câmera?

- No entanto, a partir do triângulo de pitch também podemos ver que os lados xz são influenciados por `cos (pitch)`, portanto, precisamos ter certeza de que isso também faz parte do vetor de direção. Com isso incluído, obtemos o vetor de direção final, traduzido dos ângulos de Euler de virada e inclinação:

```
direction.x = cos(glm::radians(yaw)) * cos(glm::radians(pitch));
```

```
direction.y = sin(glm::radians(pitch));
```

```
direction.z = sin(glm::radians(yaw)) * cos(glm::radians(pitch));
```

- Isso nos dá uma fórmula para converter valores de `yaw` e `pitch` em um vetor de direção tridimensional que podemos usar para olhar ao redor.

Por fim

- Configuramos o mundo da cena para que tudo seja posicionado na direção do eixo z negativo.
- No entanto, se olharmos para o triângulo de **yaw** x e z, vemos que um θ de 0 resulta no vetor de direção da câmera para apontar para o eixo x positivo.
- Para garantir que a câmera aponte para o eixo z negativo por padrão, podemos dar à virada um valor padrão de uma rotação de **90 graus no sentido horário**.
- Graus positivos giram no sentido anti-horário, então definimos o valor padrão de virada para:

yaw = -90.0f;

Alterando **pitch** e **yaw**

- Falta agora implementar a alteração dos valores de **pitch** e **yaw** através de eventos.
- Podemos fazer isso usando eventos de teclado, como em alguns jogos.

Alterando pitch e yaw

- Na função `processInput(..)` adicionaremos mais algumas opções, mapeando as teclas direcionais para girar a câmera para cima e para os lados:

```
void processInput(GLFWwindow *window)
{
    // ...

    if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
        viraCamera(0.0f, 1.0f);
    if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS)
        viraCamera(0.0f, -1.0f);
    if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)
        viraCamera(-1.0f, 0.0f);
    if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS)
        viraCamera(1.0f, 0.0f);
}
```

A função `viraCamera(..)`

- A função `viraCamera(..)` necessita algumas variáveis globais, que devem ser inicializadas com os valores mostrados abaixo

```
float sensitivity = 1.0f;  
float yaw = -90.0f;  
float pitch = 0.0f;
```

- E deve ser inicializada com valores 0, 0 antes da criação da matriz `lookAt`. Lembre-se que a posição da câmera é parte importante do cálculo da matriz. A função `viraCamera()` altera o vetor **`cameraFront`**

```
viraCamera(0.0f, 0.0f);
```

```
view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);
```

A função `viraCamera(..)`

- A função `viraCamera(..)` ajusta os valores de pitch e yaw e muda a direção da frente de câmera, o que provoca o recálculo a posição da câmera

```
void viraCamera(float x, float y)
{
    yaw  += x * sensitivity;
    pitch += y * sensitivity;

    if(pitch > 89.0f)
        pitch = 89.0f;
    if(pitch < -89.0f)
        pitch = -89.0f;

    glm::vec3 direction;
    direction.x = cos(glm::radians(yaw)) * cos(glm::radians(pitch));
    direction.y = sin(glm::radians(pitch));
    direction.z = sin(glm::radians(yaw)) * cos(glm::radians(pitch));
    cameraFront = glm::normalize(direction);
}
```

Um problema conhecido dessa abordagem é que o ângulo máximo que se pode inclinar a câmera(pitch) é 89 graus para cima (89) ou 89 graus para baixo (-89), sob pena de gerar um erro no produto vetorial.

Erguer-se ou Abaixar-se

- Erguer-se significa fazer uma **translação** da posição da câmera no **eixo y** no sentido **positivo**
- Abaixar-se significa fazer uma **translação** da posição da câmera no **eixo y** no sentido **negativo**
- Ambas ações alteram o vetor **cameraPos**

Erguer-se ou Abaixar-se

- Vamos acrescentar mais duas funcionalidades à função `processInput(..)`, mapeando as teclas Q e Z elevar ou abaixar a câmera:

```
void processInput(GLFWwindow *window)
{
    // ...

    if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS)
        cameraPos += glm::vec3(0.0f, 1.0f, 0.0f) * cameraSpeed;

    if (glfwGetKey(window, GLFW_KEY_Z) == GLFW_PRESS)
        cameraPos += glm::vec3(0.0f, -1.0f, 0.0f) * cameraSpeed;
}
```

Perceba o uso da adição vetorial para realizar a translação da posição da câmera

Mapeamento do teclado

Para **frente**

Eleva a câmera

Para **esquerda** (lateral)

Abaixa a câmera

Para **direita** (lateral)

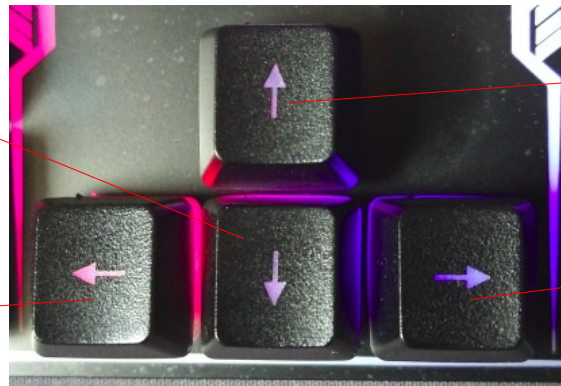
Para **trás**

Gira para **baixo**

Gira para **esquerda**

Gira para **cima**

Gira para **direita**





Exemplos

- O seguinte projeto exemplo foi disponibilizado na aula passada:
 - **GLFW_Coordenadas_3D_Cubo** :
Exemplo de visualização do cubo tridimensional

Dúvidas?



Atividade 1

- Ajuste o código do projeto **GLFW_Coordenadas_3D_Cubo**, fornecido aula passada, para que a câmera fique girando ao redor do cubo. (Código do slide 29)

Atividade 2

- Utilizando os códigos disponíveis a partir do slide 31, acrescente as funções de movimentação de câmera ao projeto

GLFW_Coordenadas_3D_Cubo